20) Experiment to understand pandas library use cases in Data science.

Program:

```
import numpy as np
import pandas as pd
data = [ [1, 'Smith', 5000],
    [2, 'Jones', 6000],
    [3, 'David', 7000],
    [4, 'Priya', 5000],
    [5, 'Ravi', 8000],
    [6, 'Anu', 3000],
    [7, 'Kumar', 6000]]
df = pd.DataFrame(data)
df.columns = ['Emp_id', 'Name', 'Salary']
print(df)
print(df.head())
print(df.tail())
df.info()
print(df['Salary'])
print("Mean:", df['Salary'].mean())
print("Median:", df['Salary'].median())
print("Mode:")
print(df['Salary'].mode())
print("Variance:", df['Salary'].var())
print("Standard Deviation:", df['Salary'].std())
print(df.describe())
print(df.describe(include='all'))
print(df.columns)
```

print(df.values)

Output:

```
   Emp_id   Name  Salary
0       1  Smith    5000
1       2  Jones    6000
2       3  David    7000
3       4  Priya    5000
4       5   Ravi    8000
5       6    Anu    3000
6       7  Kumar    6000
   Emp_id   Name  Salary
0       1  Smith    5000
1       2  Jones    6000
2       3  David    7000
3       4  Priya    5000
4       5   Ravi    8000
   Emp_id   Name  Salary
2       3  David    7000
3       4  Priya    5000
4       5   Ravi    8000
5       6    Anu    3000
6       7  Kumar    6000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Emp_id  7 non-null      int64
 1   Name    7 non-null      object
 2   Salary  7 non-null      int64
dtypes: int64(2), object(1)
memory usage: 300.0+ bytes
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: Salary, dtype: int64
Mean: 5714.285714285715
Median: 6000.0
Mode:
0    5000
1    6000
Name: Salary, dtype: int64
Variance: 2571428.5714285714
Standard Deviation: 1603.5674514745463
```

	Emp_id	Salary
count	7.000000	7.000000
mean	4.000000	5714.285714
std	2.160247	1603.567451
min	1.000000	3000.000000
25%	2.500000	5000.000000
50%	4.000000	6000.000000
75%	5.500000	6500.000000
max	7.000000	8000.000000

	Emp_id	Name	Salary
count	7.000000	7	7.000000
unique	NaN	7	NaN
top	NaN	Smith	NaN
freq	NaN	1	NaN
mean	4.000000	NaN	5714.285714
std	2.160247	NaN	1603.567451
min	1.000000	NaN	3000.000000
25%	2.500000	NaN	5000.000000
50%	4.000000	NaN	6000.000000
75%	5.500000	NaN	6500.000000
max	7.000000	NaN	8000.000000

```
Index(['Emp_id', 'Name', 'Salary'], dtype='object')
[[1 'Smith' 5000]
 [2 'Jones' 6000]
 [3 'David' 7000]
 [4 'Priya' 5000]
 [5 'Ravi' 8000]
 [6 'Anu' 3000]
 [7 'Kumar' 6000]]
```

Result:

Thus, the Pandas library was used successfully to create and explore a DataFrame and perform statistical operations on given dataset.